

LANCE : journal argumenté des corrections

Défauts observés, décisions et validation ciblée S1–S12

Projet LANCE

Muse : brouillons C01–C15 et C17–C20 ; Codex : relecture, corrections et validation

Dernière révision : 2026-10-05 14:42

Europe/Brussels (UTC+02:00)

État des runs gelé le 5 octobre 2026 à 14:09 ; code documenté à cette révision

Statut : objectif S1–S12 ouvert ; validation réelle en cours

Résumé

Ce journal suit l'objectif de terminer correctement les scénarios de développement LANCE S1–S12 sur NATO, avec `ollama-umons/qwen3.8:27b`. Les traces archivées, tests simulés et rejeux motivent quatorze corrections déployées et six correctifs locaux C15–C20. Chaque correction expose son défaut, son argumentation et sa validation disponible. Dans le lot correctif existant, S2 termine les six phases et le nettoyage sans erreur du scanner, mais son F1 confirmé de 0,688 reste perfectible ; S1 reste partiel et S12 échoue en synthèse du graphe, sans score. C17 distingue refus MQTT et panne, C18 ajoute une sonde DNS réelle, C19 exige une preuve MQTT native au-delà de HTTP 101. C20 ajoute une sonde native liée avant de résoudre un service incertain ; C17–C20 ne sont pas encore validées sur NATO. Le suivi planifié reste désactivé ; aucun lot identique n'est répété, aucun déploiement n'a lieu pendant le lot actif et aucune vérité terrain n'est modifiée pour améliorer les scores. Ce document reste un journal de travail, pas une annonce de réussite des douze scénarios.

Table des matières

1	Objectif et périmètre	3
2	Méthode de diagnostic et statut des preuves	3
3	Constats et corrections C01–C15	4
3.1	Cohérence de la projection et du résumé : C01 et C03	4
3.2	Sondes, alias et périmètre de découverte : C02, C05 et C06	5
3.3	Portée locale des commandes : C07, C10 et C15	5
3.4	Preuves HTTP, divulgation et liaison native : C04, C09 et C14	6
3.5	Délais, reprise et fixtures : C08, C11, C12 et C13	7
4	Échec S12 du lot correctif : un résultat ouvert	8
5	C16 : rendre la troncature diagnostiquable	9
6	Nouvelles corrections ciblées C17–C20	9
6.1	C17 : refus explicite d’authentification MQTT native	9
6.2	C18 : couverture DNS par dns-nsid	9
6.3	C19 : HTTP 101 isolé, hypothèse WebSocket MQTT	10
6.4	C20 : suivi natif MQTT sur WebSocket	11
7	Plan ciblé de validation	11
8	État de chaque scénario au gel du 5 octobre à 14 :09	13
9	Discussion et limites	13
10	Conclusion	14
	Références et pièces primaires	14
A	Retrouver chaque correction	15

1 Objectif et périmètre

Ce rapport est le journal argumenté des corrections logicielles apportées à LANCE après la campagne réelle Qwen UMONS S1–S12. Il ne réécrit pas le rapport de référence, qui conserve les résultats historiques et l’analyse des runs initiaux [1] : il documente, correction par correction, ce qui a déclenché chaque changement, quel défaut a été établi, ce que le code faisait avant et fait après, pourquoi la correction est justifiée au-delà du cas observé, ce que les tests et rejeux hors réseau ont consigné, et quelle validation réelle reste en attente. Le gel expérimental de ce bilan est le 5 octobre 2026 à 14 :09, Europe/Brussels (UTC+02 :00), postérieur au gel du rapport de référence et intégrant le premier retour du lot correctif.

Le périmètre est S1–S12, scénarios de développement, sur NATO et pve-nato exclusivement ; le homelab personnel est hors périmètre. L’ancien lot initial s’est terminé naturellement malgré l’échec d’arrêt authentifié (HTTP 401) ; les démarrages S13–S19 qui ont suivi restent archivés mais exclus des résultats. Quatorze corrections sont déployées sur 0f4e5ff, après la CI réussie du SHA exact (3 213 tests, un ignoré). C15–C20 restent locales : elles concernent une inspection statique refusée, les diagnostics de génération, les refus MQTT, la couverture DNS et les conclusions WebSocket et la sonde native qui les vérifie. L’objectif courant est que S1–S12 achèvent leurs six phases, avec livrables validés, absence d’erreur de phase masquée et nettoyage terminé. La validité sémantique des preuves et les faux négatifs sont examinés séparément ; un statut complet ne signifie ni F1 parfait ni absence de vulnérabilité. Le suivi est **PAUSED**. Les nouveaux runs viseront seulement un défaut nommé, après analyse du lot déjà lancé et vérification de la CI et du SHA réellement déployé.

2 Méthode de diagnostic et statut des preuves

Le diagnostic repose sur trois sources primaires : le code inspecté en lecture seule dans l’espace de travail, les archives de runs aux empreintes vérifiées, et le faisceau **evidence.json** du dossier de corrections, qui consigne pour chaque correction le commit, les chemins modifiés, les artefacts de rejeu, la validation hors réseau et la validation réelle exigée [2, 3]. La rédaction Muse a utilisé les archives en lecture seule. Codex a consulté l’état de l’API NATO et exécuté des tests logiciels simulés pour C16–C20 ; aucun nouvel audit ni appel réel au modèle n’a été lancé dans cette révision. Le code préparé reste local, sans déploiement pendant le lot actif.

La méthode distingue strictement deux niveaux de preuve. D’une part, les contrôles logiciels et les rejeux hors réseau : tests ciblés du commit, rejeux de livrables archivés à empreintes vérifiées, comparaisons de verdicts et de références de preuves à vérité terrain non chargée et scores non recalculés. Ces contrôles vérifient les cas couverts et les changements observés dans les populations rejouées, mais ils ne prouvent ni la finalisation d’une analyse en laboratoire ni un gain expérimental. D’autre part, la validation réelle : nouveaux audits sur NATO après déploiement d’un commit exact à CI vérifiée, sous le contrat courant. Au snapshot, les tests logiciels sont établis ; les observations réelles du lot correctif restent partielles. S12 est en échec, S1 est partiel et S2 a terminé ses six phases ; leurs états sont distingués dans le tableau actualisé. Les critères d’acceptation des corrections restent à conclure.

Les vrais positifs (TP) sont les prédictions appariées à une référence, les faux positifs (FP) celles sans appariement retenu, et les faux négatifs (FN) les références non retrouvées. Le F1 est la moyenne harmonique de précision et rappel, soit $2TP/(2TP + FP + FN)$; ici seul l’entonnoir confirmé est cité. Un FP peut être un constat soutenu hors référence : cette étiquette de mesure ne suffit pas à établir sa fausseté sémantique.

Les codes de scénarios désignent les configurations du benchmark. Une *fixture* est un composant préparé du laboratoire ; une *projection* est la représentation finale regroupant les constats. *Runner* désigne la machine d’exécution, *rejeu* l’interprétation hors réseau d’octets archivés. CI signifie

intégration continue ; SHA identifie un commit Git, c'est-à-dire une révision de code. Les contrats de mesure et de preuve structurent la lecture. L'historique (lot initial et S1 commun de référence) est immuable sous `strict-v3.14` / `evidence-v13` : scores conservés tels quels, références terrain non modifiées, absences non comblées. L'étape `strict-v3.15` / `evidence-v14` a été préparée comme intermédiaire mais n'a jamais été déployée et ne doit pas être présentée comme un état observé. Le contrat actuel des runs correctifs est `strict-v3.16` / `evidence-v15` ; leurs résultats devront porter ce contrat et ne seront jamais comparés à contrat constant avec l'historique. Aucun changement de la vérité terrain ne sera introduit pour améliorer des scores : les scores manquants restent indisponibles, jamais zéro.

3 Constats et corrections C01–C15

Les quinze corrections sont numérotées C01 à C15 dans l'ordre de leur préparation et regroupées ci-dessous par cause. Chacune suit le même schéma de lecture : déclencheur réel observé dans les archives, défaut établi, comportement avant et après, justification générale, tests et rejeux consignés, et résultat réel encore en attente. Les causes concernent le pipeline, les sondes, les frontières de portée, les preuves et les fixtures de laboratoire.

3.1 Cohérence de la projection et du résumé : C01 et C03

C01 (`eb3b865`) part d'un double déclencheur sur S2 : le résumé final de phase 4 conservait des compteurs de candidats et d'ignorés antérieurs au suivi de découverte, et la sonde d'identifiants MySQL à mot de passe vide dégénérait en invite interactive (`-p`), passait une redirection shell comme argument de base et exigeait une permission système inutile. Le défaut est double lui aussi : un résumé qui survit à la projection qu'il résume, et une construction de commande client qui ne force pas le mode non interactif en TCP (transport réseau par connexion). Avant, le résumé affichait 29 candidats et zéro ignoré pour une projection de 30 candidats, dont 29 testés et un ignoré, tandis que la sonde MySQL vide pouvait se bloquer ou viser la mauvaise base ; après, le résumé est recalculé à partir de la projection finale, incluant l'identifiant non testé ajouté par le suivi, et les options client documentées imposent le TCP avec mot de passe vide explicite [3]. La justification dépasse S2 : tout résumé conservé après une phase de suivi peut mentir sur ce qui a été testé, et toute sonde d'identifiants doit être non interactive par construction. Hors réseau, 177 tests passent et le rejeu de la phase 4 de S2 conserve tous les verdicts et références de preuves. Le S2 correctif archivé termine ses six phases et contient un reçu MySQL authentifié frais ; ses 35 contrôles arithmétiques passent. Il exerce C01 sans isoler son effet des autres changements. Les critères encore ouverts doivent être examinés dans cette archive avant de motiver une éventuelle relance, jamais par défaut.

C03 (`f2c4ac6`) a été déclenchée par la disparition de VULN-013, constat MQTT (messagerie de publication/abonnement) vérifié de S3, de la projection canonique finale après une analyse redondante de deux hôtes publics déjà connus. Le défaut est un suivi de découverte qui réanalyse sans exclure les adresses déjà connues, puis remplace la file vérifiée par le résultat de la seconde passe. Avant, la projection historique ne conservait que 38 tests et 26 confirmations ; après, les adresses connues sont exclues du suivi et le rejeu hors réseau reconstruit la file d'origine à 39 identifiants planifiés, avec 39 tests et 27 confirmations dont VULN-013 et ses preuves d'outil d'origine. La règle générale est qu'une redécouverte ne doit jamais détruire une vérification planifiée : le connu s'exclut, le planifié se conserve. Les contrôles consignés comptent 38 tests ciblés, complétés après un raffinement sur adresses malformées. La validation réelle exigée est un run S3 après déploiement, avec contrôle que tous les identifiants planifiés figurent dans la projection finale.

3.2 Sondes, alias et périmètre de découverte : C02, C05 et C06

C02 (31501fe) répond à un gaspillage observé sur S2 et S3 : le scanner tentait des négociations HTTP vers WebSocket après qu’une sonde nmap fraîche et réussie eut signalé le port optionnel 9001 fermé. Le défaut est l’absence de lecture, par le scanner, du résultat de sa propre sonde préalable sur un port optionnel non déclaré. Avant, des sondes redondantes étaient émises vers des écouteurs fermés ; après, les écouteurs fermés omettent ces sondes tandis que les écouteurs ouverts conservent les leurs. La justification est une discipline générale des sondes : une observation fraîche de fermeture doit inhiber les tentatives applicatives redondantes sans réduire la couverture de découverte. Hors réseau, 110 contrôles ciblés passent et cinq rejeux de traces de brokers conservent des constats identiques. Le S2 correctif archivé exerce C02 et termine sans erreur de scanner ; son effet individuel n’est pas isolé. S3 est encore en cours au gel. La couverture et l’absence de sonde redondante sont à contrôler dans ces archives avant toute nouvelle relance.

C05 (a7136e5) corrige un diagnostic erroné sur S4 : le nom de service Nmap `mbap`, normalisé mais sans alias Modbus, produisait un diagnostic de service non pris en charge en double malgré la sonde Modbus déclarée sur le même port. Avant, deux diagnostics coexistaient pour un seul service ; après, `mbap` et `modbus` fusionnent sur même port et transport, et le diagnostic erroné disparaît sans revendiquer de gain de couverture. La règle générale est que la normalisation des noms doit s’accompagner de leurs alias protocolaires avant tout diagnostic d’incompatibilité. Le rejeu archivé de l’automate programmable (PLC) conserve appels et constats identiques, avec 36 contrôles ciblés. Reste à valider le statut de phase 3 de S4 après déploiement, le run historique demeurant inchangé.

C06 (9d7b285) traite une auto-analyse du runner sur S4 : l’adresse IP cliente retournée par `USER()` MySQL a été traitée comme une cible nouvelle et a consommé une analyse complète. Avant, le suivi incluait le runner dans les cibles ; après, les interfaces du runner sont exclues de la découverte tandis qu’un hôte synthétique réellement nouveau reste conservé dans le rejeu. La justification est une hygiène générale de la découverte : l’infrastructure d’audit ne fait jamais partie de la surface auditée, mais l’exclusion ne doit pas fermer la porte aux vraies nouveautés. Les contrôles consignés comptent 31 tests, avec rejeu des 32 livrables de phase 4 de S4 (recherche d’interfaces simulée depuis l’adresse enregistrée, aucun rerun réel simulé). La validation réelle exige un run S4 après déploiement, avec exclusion du runner et préservation d’une découverte authentique.

3.3 Portée locale des commandes : C07, C10 et C15

C07 (ba14300) répare un faux refus sur S4 : l’inventaire binaire local de phase 5 (une commande composée d’inspection de fichiers) était refusé parce que ses arguments de recherche contenaient des noms de programmes réseau. Le garde de portée confondait le vocabulaire des arguments avec la destination de l’action. Avant, l’inventaire légitime était bloqué ; après, la décision de portée examine la destination réelle de la commande et l’inventaire statique local est admis, tandis que les refus de portée pour actions réseau restent dus. La justification est générale : un garde de portée doit juger ce qu’une commande touche, pas les mots qu’elle contient. Hors réseau, 44 tests passent et le rejeu de 266 enregistrements de phase 5 issus de cinq runs ne modifie que cet inventaire S4, sans exécuter aucune commande. Un run S4 après déploiement reste nécessaire.

C10 (43e519c) étend la même doctrine aux chemins locaux : sur S2 et S7, des composantes du chemin `.ssh/` étaient faussement reconnues comme des commandes réseau SSH. Avant, une inspection locale d’un répertoire `.ssh/` était refusée comme action réseau invérifiable ; après, les composantes d’une arborescence contenant `.ssh/` sont traitées comme locales. La limite est explicitement conservée : la recherche du seul nom `.ssh`, sans barre oblique, reste soumise au garde historique, car un nom isolé n’établit pas à lui seul un chemin local. Hors réseau, 50 contrôles passent et le rejeu de politique sur 475 enregistrements de huit archives ne modifie que deux refus, sans exécuter de commande. La validation réelle concerne S2 et S7, avec condition d’arrêt authentifié, inactivité, nettoyage et CI du commit exact.

C15 (74fcef4) prolonge C10 d'un cran resté ouvert : l'appel statique `find` avec critère `-name .ssh` sur S9 était encore classé comme action réseau invérifiable. Le déclencheur est l'unique admission manquée relevée par le rejeu : sur 186 appels shell de phase 5 contrôlés hors réseau, seule la trace `tc-538baa132184453c8487cee6953da0b2` de S9 change d'admission, passant de `intrusion_command_destination_unverifiable` à admise. Avant, une recherche statique de clés locales était bloquée ; après, elle est admise comme inspection locale, sans exécution d'outil, sans lecture de vérité terrain et sans score recalculé. La justification générale est la même qu'en C10, appliquée au motif sans slash : un `find` statique de noms de clés est une inspection locale, pas une action réseau. L'admission suit une grammaire positive de chemins littéraux et prédicats de recherche ; `-exec`, `-execdir`, `-ok`, `-delete`, les substitutions et les options inconnues restent refusés. Les contrôles consignés comptent 141 tests ciblés. Le statut est particulier et doit rester tel quel : C15 est préparée seulement, non poussée ni déployée. Son SHA d'origine pour le rejeu est `e1e6901`, et après rebase sur la migration documentaire les trois blobs (source de portée, tests, documentation de phase) sont identiques. Elle ne donne aucun gain d'accès ni de score, et sa validation réelle (un run S9) attend la fin du lot correctif avec nettoyage, CI et déploiement vérifiés. Le lot actif reste sur `0f4e5ff` et ne valide que ses quatorze corrections déployées.

3.4 Preuves HTTP, divulgation et liaison native : C04, C09 et C14

C04 (44add6c) corrige le rejet de deux constats HTTP de S3 (VULN-025 et VULN-027), dont les listes de chemins séparés par virgules figuraient dans un endpoint singulier alors que des réponses fraîches valides existaient. Le défaut est une normalisation qui rejetait comme non primaires des réponses valides au lieu d'extraire le premier chemin principal en conservant la liste. Avant, deux constats dotés de réponses fraîches étaient rejetés pour leur forme ; après, la forme composite est normalisée sans dispenser chaque preuve de sa liaison à un chemin effectivement observé, et les contrôles d'hôte, de port et de chemin primaire restent stricts. La règle générale est qu'une liste de chemins dans un champ singulier est un défaut de forme à normaliser, pas une preuve à rejeter. Hors réseau, 62 contrôles passent et le rejeu de 102 vérifications existantes (deux diagnostics S1, S2, S3) ne modifie que les deux verdicts S3 visés, sur leurs références de preuves d'origine. La validation réelle passe par un run S3 après déploiement.

C09 (ae7abdf) traite la divulgation de chemins sous deux angles opposés : sur S4, la publication fraîche de chemins par `robots.txt` était rejetée par un vérificateur limité à la divulgation de version ; sur S6, une simple bannière `nginx` sans endpoint était à tort créditée comme exposition de chemins. Le défaut est un contrat de preuve qui ne distinguait pas version et chemins. Avant, un constat de chemins était rejeté et un crédit de référence non justifié subsistait ; après, une preuve dédiée des chemins exige une réponse fraîche liée à la ressource créditée, y compris pour les fichiers secondaires, tandis que les règles des opérations couplées restent inchangées. La justification est qu'une divulgation de chemins ne se prouve que par le contenu du fichier qui les publie, jamais par une bannière. Hors réseau, 474 contrôles passent et 197 interprétations de preuves d'origine, sur sept archives vérifiées, conservent toutes leurs références ; seule la correction change le verdict S4, et l'appariement sur les populations confirmées retire le crédit V14 de S6 sans recalculer aucun score officiel. Cette correction prépare le contrat `strict-v3.15 / evidence-v14`, étape intermédiaire qui n'a jamais été déployée. La validation réelle exige des runs S4 et S6 après déploiement vérifié, avec enregistrement du nouveau contrat et sans comparaison à contrat constant.

C14 (0f4e5ff) est la correction native la plus large : sur S8 à S11, l'endpoint composite FTP bloquait les preuves exactes de lecture de répertoires et fichiers, les lectures natives Redis et SNMP étaient bloquées par des valeurs de substitution HTTP racine, un datagramme SNMP arbitraire était accepté comme preuve de lecture de communauté avec une description candidate périmée, et les réponses natives manquaient de bornes de destination, d'authentification et de propriété. Avant, des lectures réellement archivées (secrets Redis, dumps et configuration FTP, réponse SNMP `public`) étaient rejetées par une liaison au faux endpoint, ou des descriptions périmées (`private`,

écriture) survivaient ; après, les chemins FTP natifs, les lectures Redis typées et les lectures SNMP typées (réponse binaire BER valide liée à la requête, à l'identifiant d'objet OID et au pair réseau, sans inférer un droit d'écriture d'une lecture) lient chaque preuve à sa ressource, sa destination et son authentification. La justification générale est la liaison native : une preuve vaut pour la ressource exacte qu'elle a touchée, avec le protocole, la destination et les droits réellement exercés. Hors réseau, 289 tests passent et le rejeu pur de 228 confirmations sur treize archives vérifiées ne modifie exactement que neuf admissions sur S8-S11, en comparant `b39b0b1` préparé à `0f4e5ff` et non au baseline `e9d7d55` ; la vérité terrain n'est pas chargée et aucun score n'est recalculé. Ces neuf admissions ne sont ni des vrais positifs mesurés ni des gains : aucune ne doit être présentée comme un faux négatif résorbé ou une progression. Cette correction porte le contrat actuel `strict-v3.16 / evidence-v15`, avec `git diff -check` passé et CI 37290434113 réussie ensuite. La validation réelle exige des runs S8 à S11 après déploiement sous ce contrat.

3.5 Délais, reprise et fixtures : C08, C11, C12 et C13

C08 (`da6e607`) clarifie l'échec temporel de S5 : le routeur et le contrôleur de chauffage/ventilation (HVAC) avaient épuisé le délai de 240 secondes par appareil, mais la finalisation exposait une erreur générique de la bibliothèque cliente (SDK) comme erreur fournisseur terminale. Le défaut est un diagnostic qui ne distinguait pas l'épuisement d'un budget de temps d'une panne du fournisseur. Avant, la cause terminale indiquait `providererror` ; après, elle indique `deadline`, tandis que l'observation d'erreur d'origine et la requête sont préservées. La justification est une honnêteté générale des diagnostics : classer correctement un délai ne rallonge aucun budget et ne garantit en rien la conclusion du modèle, qui peut tout aussi bien échouer à nouveau. Hors réseau, 135 contrôles passent avec rejeu des frontières, dont un contrôle de délai précoce qui reste `providererror`. La validation réelle est un run S5 après déploiement, avec rapport honnête des analyses incomplètes.

C11 (`d8a9706`) remplace une fixture S3/S7 incapable de tenir sa promesse : l'utilitaire privilégié censé implémenter une lecture root était un script Bash marqué SUID (exécution avec l'identité du propriétaire), alors que Linux ignore les bits set-UID sur les scripts. Avant, la lecture privilégiée déclarée ne pouvait pas fonctionner ; après, un utilitaire compilé en lecture seule l'implémente, avec cas Ansible limités aux faits et assertions et sans exécution locale de scénario. La justification est qu'une fixture doit être réellement capable du comportement qu'elle simule, sinon les runs testent un décor. Les contrôles consignés comptent 72 tests ciblés, complétés par les vérifications de syntaxe du compilateur, de Bash et d'Ansible. La validation réelle porte sur S3 et S7 sur NATO (comportement compilé, permissions réelles), car les contrôles logiciels ne certifient pas la vérité terrain.

C12 (`e13d2e1`) répare la fixture Node-RED de S10 sous deux aspects : le littéral d'octets du gabarit contenait de l'Unicode invalide qui empêchait la compilation, et les échecs de disponibilité ainsi que les contrôles d'administration manquants ne bloquaient pas le précontrôle global. Avant, le scénario pouvait tourner sur une fixture non prête ; après, les octets sont corrigés et les contrôles de disponibilité bloquent le précontrôle. La justification est qu'un précontrôle qui laisse passer une fixture non prête invalide tout ce qui suit. Hors réseau, 81 contrôles logiciels passent (arbre syntaxique Python (AST), syntaxes Bash et Ansible, aucune exécution locale de scénario), avec CI 37284944781 réussie sur le SHA exact (3 159 tests, un ignoré, jobs de déploiement ignorés). Le S12 correctif passe déploiement, injection et vérification avant d'échouer dans le graphe ; cela étaye la préparation, pas la réussite de l'audit ni la validation complète Node-RED. L'exercice des propriétés Node-RED sur S10 reste attendu.

C13 (`b39b0b1`) répond à l'échec de reconnaissance du S12 initial : la reconnaissance complète revenait prématurément après une seconde réponse sans outil sur motif `length` alors que la couverture minimale manquait (15 sondes pour 35 nœuds déclarés, puis deux réponses vides et arrêt au tour 5, bien avant le plafond de 50 tours). Le défaut est un retour anticipé alors que le profil complet n'activait pas la continuation stricte disponible dans le contrôleur. Avant, le

fournisseur rendait prématurément la main et le validateur de couverture signalait l'échec ; après, la continuation bornée persiste dans la limite des budgets existants, sans élargir le budget modèle, le contexte, les tours ni les plafonds, et sans assurer la convergence du modèle. La justification est qu'un arrêt sur réponse vide ne vaut ni couverture ni échec franc : la reconnaissance doit soit couvrir le minimum, soit échouer proprement. Hors réseau, 165 tests ciblés passent (longueurs vides, textes ordinaires, sauvegardes rejetées, blocages persistants bornés, arrêt utilisateur, suites existantes) avec `git diff -check` passé et CI 37287309302 réussie, sans aucune sonde réseau exécutée. La validation réelle exigeait un run S12 complet en six phases sur NATO après repos, nettoyage et CI du commit exact ; l'échec historique reste inchangé et aucune observation manquante n'est inférée. Le résultat effectivement obtenu est décrit à la section suivante, et il montre que ce correctif n'a pas encore été exercé par un run qui échoue avant de l'atteindre.

4 Échec S12 du lot correctif : un résultat ouvert

Le lot correctif `batch-a4a0a1d88e00`, explicitement borné à S12, S1 puis S2–S11 avec les six phases, le profil `full` et le nettoyage automatique, tourne sur le commit déployé `0f4e5ff` sous contrat `strict-v3.16 / evidence-v15`. Son premier résultat, consigné dans `corrective-s12-graph-failure-review.json`, est un nouvel échec de S12 qui doit être lu avec précision : il échoue en phase 1 (analyse du graphe), alors que le S12 initial avait passé la phase 1 et échoué en phase 2. Les conditions et la phase d'échec diffèrent, et aucune comparaison directe ne vaut.

Le run `2026-10-05_114742` rapporte une génération de phase 1 tronquée (`finish_reason=length`) sans sauvegarde validée, puis l'échec de la récupération bornée en écriture seule (`missing_validated_deliverable`, réponse de récupération elle aussi tronquée après deux tentatives). Les phases 2 à 6 sont sautées sur prérequis manquants, le nettoyage est terminé, et le score officiel reste indisponible (HTTP 404) : cet échec n'est pas un zéro. Les observations consignées comptent 35 nœuds, 35 nœuds couverts par des observations enregistrées, 7 réponses dont 6 sur motif `length`, 4 appels d'outils exécutés et aucune sauvegarde. Au gel de 14 :09, S1 correctif est terminé mais partiel, S2 est terminé et S3 est en phase 3. Le rapport initial, gelé à 11 :32, conserve ses propres tables historiques.

L'inspection logicielle sur `0f4e5ff` vérifie les garde-fous (seule `save_deliverable` exposée en récupération, refus des sauvegardes proposées par une réponse tronquée même analysable, couverture exigeant une observation par nœud déclaré, invite de reprise conservant les sept rubriques, gardes d'arrêt et de budget inchangés), mais elle établit seulement la troncation et l'absence de sauvegarde validée. Le budget initial est de 4 096 jetons par réponse et la récupération repose sur une valeur par défaut de 2 048 jetons, dont les variables effectives d'environnement n'ont pas été archivées dans ce run : cette valeur par défaut ne peut donc pas être affirmée comme plafond vérifié de la requête. Surtout, ni le raisonnement privé du modèle ni une capacité maximale de contexte ne sont mesurés ou inspectés ici : attribuer la troncation à l'un ou à l'autre serait une inférence non fondée, et ce rapport s'y refuse. Il est également établi que le correctif de continuation de reconnaissance C13 n'a pas été exercé par ce run, qui échoue en phase 1 avant toute reconnaissance, et que la correction de portée C15 lui est sans rapport. La suite proposée est une calibration de la génération et de la récupération de phase 1 sur les données de grande topologie enregistrées, avec compteurs exacts de requêtes et sorties et configuration figée, en préservant la validation stricte des sauvegardes ; toute modification d'un plafond de sortie devra être divulguée et validée dans un vrai rerun S12 sur NATO, sans substitution narrative déterministe ni score rétroactif. En attendant la fin du lot actif, aucune relance ni déploiement ne doit intervenir.

5 C16 : rendre la troncature diagnostiquable

Cette correction supplémentaire a été préparée par Codex après le snapshot Muse. L’archive S12 signale six réponses `length`, mais le journal durable ne conserve pas les limites effectivement envoyées ni les compteurs par réponse. Le code sait déjà extraire ces métadonnées ; elles n’étaient pas transmises au journal filtré. Sans elles, une augmentation de budget serait une expérience insuffisamment contrôlée.

Le correctif local `08f2837` ajoute au schéma additif `model.obs1` les plafonds de tours et de sortie au début d’une invocation, le plafond de sortie à la frontière de chaque requête envoyée et les compteurs disponibles de jetons d’entrée, de sortie et de raisonnement, ainsi que les longueurs en caractères du raisonnement et du texte visible. Les réponses de repli après erreur de génération sont couvertes. Seuls des entiers bornés sont persistés. Un compteur de réponse inconnu ou invalide reste `null` ; une limite de requête ou d’invocation absente ou invalide est omise, sans valeur inventée. Aucun texte du raisonnement, prompt, argument d’outil ou corps de réponse n’entre dans ce journal.

Neuf nouveaux cas de régression échouent avant la correction. Après celle-ci, 189 tests ciblés passent : diagnostic, récupération du graphe, délais, compatibilité des requêtes et finalisation. Une comparaison simulée avec et sans observation conserve exactement les arguments de la bibliothèque cliente et le texte retourné. `git diff -check` réussit. Le correctif n’augmente aucun budget, n’admet aucune sauvegarde tronquée, ne change aucun score et n’est ni poussé ni déployé au gel de cette révision. Son rôle est de rendre le prochain diagnostic S12 vérifiable ; il ne constitue pas une résolution de l’échec de synthèse. La taille en caractères ne se convertit pas en jetons, et le plafond envoyé par LANCE ne mesure pas le plafond du serveur. Voir `generation-diagnostics-correction.json` et le journal des tests.

6 Nouvelles corrections ciblées C17–C20

6.1 C17 : refus explicite d’authentification MQTT native

Le déclencheur est un faux statut d’erreur : des refus natifs explicites du courtier MQTT, authentification requise, étaient comptés comme erreurs de processus dans le scanner et le suivi d’exécution. Le défaut établi dans `mqtt-auth-outcome-correction.json` est cette confusion entre accès négatif et panne d’outil. Avant, un code entier 5 avec interprétation `connection_refused_auth_required` basculait en `failed` ; après, seule la conjonction stricte — code entier 5, interprétation exacte de l’enveloppe, sortie standard vide et refus natif explicite — sort du statut d’erreur, tandis que erreurs explicites, délais et annulations restent des échecs. Un refus reste donc un résultat négatif, jamais un constat `no_auth`. La justification est générale : un garde de fiabilité doit juger la cause attestée, pas l’absence de données applicatives. Avant correction, trois tests de frontière de l’exécuteur et du journal, avec processus simulé, échouent ; après, 138 tests passent dans une invocation ciblée et 13 dans le module travailleur, en deux invocations séparées, soit 151 cas distincts ; les invocations mixtes à erreurs de fixtures ne sont pas comptées comme validations. Le jeu hors réseau porte sur 4215 enregistrements de registre et 1609 empreintes de fichiers archivés vérifiées, avec 33 reclassifications — 1 en phase 2, 8 en phase 3, 24 en phase 4 — sans nouvel appel réseau ou modèle, sans chargement de vérité terrain et sans score recalculé. La validation réelle exigée est un courtier à anonymat refusé, S8–S11 ou équivalent NATO après repos, avec CI du SHA exact et SHA déployé vérifiés, en inspectant séparément reçus de refus, erreurs du scanner et complétion des phases.

6.2 C18 : couverture DNS par dns-nsid

Le déclencheur, consigné dans `dns-scanner-correction.json`, est le diagnostic d’absence de règle pour le service `domain` sur `s1-router` sur le run 2026-10-05_120819, référence `tc-`

`bab45eb3d5ec4089821950a5b0350a50`, port 53 en `tcp`. Le défaut est l'absence de règle de scanner pour les services `domain` et `dns`. Avant, aucun sondage DNS n'était planifié ; après, ces services sont associés à la règle Nmap `dns-nsid`, au port et transport exacts demandés, avec balayage UDP seulement si UDP déclaré ou observé. Aucun constat automatique, aucune récursion ni transfert de zone ne sont ajoutés ; transports inconnus, indisponibilité d'outil et pannes réseau restent des erreurs diagnostiques, et un transport absent, vide ou nul reste refusé par le scanner dans le complément validé. La normalisation Recon historique peut néanmoins fournir TCP par défaut en amont : cette règle ne certifie pas seule la provenance du transport absent. La source lue est le script `dns-nsid.nse` [6], consulté le 2026-10-05 dans sa portée `NSID` et règle de port TCP et UDP, sans reprise de récursion ni de transfert. La règle vaut au-delà de S1 : un nom de service normalisé exige son alias protocolaire avant tout diagnostic d'incompatibilité. Avant, 10 régressions échouent ; après, 36 tests ciblés passent. La validation réelle exigée est S1 après nettoyage du lot, CI du SHA exact et déploiement vérifié, avec inspection des requêtes et résultats DNS. Limites : aucun run DNS NATO réel, un code retour 0 sans texte `NSID` ne prouve pas un service sûr, aucun changement de terrain ni de score historique, et le port 9001 `tor-orport` reste un diagnostic ouvert.

6.3 C19 : HTTP 101 isolé, hypothèse WebSocket MQTT

Le déclencheur est une surinterprétation du scanner : une réponse HTTP 101 isolée était conclue comme accès MQTT anonyme confirmé, alors que le vérificateur natif rejette déjà les preuves purement HTTP. Le cas observé est le run `2026-10-05_120819`, référence `tc-0b29c137e3f74add9d3a87a813a99641`, documenté dans `mqtt-ws-scanner-evidence-correction.json`. Avant, HTTP 101 suffisait à confirmer ; après, HTTP 101 seule ne conserve qu'une hypothèse `suspected` à IP, port et chemin exacts, et seule une preuve native liée — reçu MQTT sur WebSocket de confiance avec `CONNACK` et `SUBACK` anonymes liés et `PUBLISH` complet apparié en IP, port, chemin et sujet — peut confirmer. Aucun alias Tor vers MQTT ni suppression du diagnostic de service non pris en charge, aucune nouvelle sonde native dans cette correction ; `tor-orport` reste un diagnostic ouvert. La justification est la liaison native : HTTP prouve HTTP, jamais MQTT applicatif. Avant, 4 échecs pour 46 passants ; la suite finale exécutée par Codex compte 152 passants, après ajout des contrôles de transport DNS absent et de destination finale HTTP/redirection, les comptages se recouvrant entre modules. Le rejeu hors réseau de 124 fichiers d'analyse ne modifie que 4 confirmations en hypothèses — S1 commun, S1 correctif, S6 et son doublon `discovered` — sans appel réseau natif, sans vérité terrain chargée, sans octet d'archive modifié et sans score recalculé. La correction rétrécit la conclusion sans ajouter de planification d'appels natifs ; elle ne résout pas le statut partiel du cycle de vie S1. La validation réelle exigée est S1 après nettoyage, CI et déploiement vérifiés, en préservant l'hypothèse HTTP, une preuve applicative MQTT fraîche et les diagnostics de phase 3 si non résolus.

Méthode et limites transversales C17–C19. Tests ciblés et rejeux hors réseau vérifient les cas couverts et les populations rejouées ; ils ne prouvent ni finalisation en laboratoire ni gain expérimental. Aucun résultat réel de C17–C19 n'existe encore et les trois correctifs restent préparés localement sans déploiement pendant le lot actif. Le premier run du lot dont les six phases sont achevées sans erreur de phase, décrit dans `corrective-s2-completion-review.json` et `s1-s12-targeted-validation-matrix.json`, est S2 en `2026-10-05_124823` : six phases et nettoyage terminés, zéro erreur de scanner, arithmétique 35/35, entonnoir confirmé *F1* 0,688 avec 11 vrais positifs, 8 faux positifs et 2 faux négatifs, 6 constats hors référence et 2 redondants, sans aucune perfection de vérité terrain. Ce run multi-changements exerce C01 et C02 sans isoler l'attribution à une correction unique. S1 reste partiel, S12 en échec sans score et S3 en cours ; budgets et modèle de `0f4e5ff` sont inchangés, contrats correctifs `strict-v3.16` et `evidence-v15` sans comparaison à contrat constant avec l'historique `strict-v3.14` et `evidence-v13`. Ce journal est en cours, pas un bilan final des douze scénarios : un refus MQTT ne doit pas devenir une panne effacée, ni HTTP 101 une preuve MQTT.

6.4 C20 : suivi natif MQTT sur WebSocket

Le cas déclencheur est S1, run 2026-10-05_120819. La trace est celle de C19 (section 6.3). Une négociation WebSocket complète sur l’endpoint existant du scanner MQTT restait sans suite native, et le libellé Nmap incertain demeurait non couvert. C20 complète C19 sans l’invalider : HTTP 101 isolé reste une simple hypothèse, jamais une confirmation MQTT.

Avant, aucune sonde applicative n’était planifiée après la poignée de main ; après, une vraie sonde `mqtt_ws_listen` avec `count 1` et `timeout 5` est planifiée après une réponse `http_request` fraîche et faisant autorité de phase 3, à `ip`, `port` et `path` exacts, avec `final_url` sans redirection, en-têtes `Upgrade`, `Connection` et version 13, et `Sec-WebSocket-Accept` lié à la clé réellement envoyée. La clé initiale du scanner est fixe : elle ne doit pas être décrite comme un nonce aléatoire frais. Le nom incertain n’est résolu qu’après une preuve applicative complète acceptée par `trusted_mqtt_ws_messages` apparié au même port `tcp`, avec nom d’origine et base de résolution conservés. Aucun alias Tor vers MQTT. Sans contexte MQTT connu ou avec `Upgrade` incomplet, aucune requête n’est émise ; refus, silence, outil absent, panne ou autre destination restent des diagnostics.

La justification est la liaison native déjà invoquée en C14 et C19 : HTTP prouve HTTP, seul un échange MQTT complet prouve MQTT, et un libellé incertain ne se résout que par preuve applicative appariée, sans alias arbitraire.

Avant correction, le sous-ensemble initial de 18 cas compte 11 échecs pour 7 réussites ; après, 183 tests passent, dont 31 nouveaux cas, les suites se recouvrant avec C17–C19. Le jeu hors réseau porte sur 124 fichiers d’analyse et 1609 empreintes vérifiées et propose 4 requêtes, S1 commun, S1 correctif, S6 et doublon, sans aucun appel réseau ni résultat applicatif inféré, sans vérité terrain chargée ni score recalculé.

Limites : correctif local non déployé, aucun vrai S1 corrigé observé ; charge et durée supplémentaires à mesurer, budgets `full` inchangés. Validation réelle exigée : S1 après nettoyage, CI du SHA exact et SHA déployé vérifiés. Référence primaire : RFC 6455 [7], décembre 2011, sections 1.3, 4.1 et 4.2.2, consultées le 5 octobre 2026. Le commit local est `e6550fd` ; l’argumentation et les journaux sont dans `mqtt-ws-native-scanner-coverage-correction.json`.

7 Plan ciblé de validation

La réussite attendue porte sur les six phases demandées, les sauvegardes validées, les erreurs de phase et le nettoyage, puis sur la lecture honnête des preuves. Elle ne repose pas sur `total_tool_errors=0` : une requête rejetée par un garde peut être correctement enregistrée sans rendre toute la phase incomplète. Chaque erreur reste examinée. Aucun statut historique, critère de vérité terrain ou score manquant n’est réécrit. S2 a satisfait le critère de cycle de vie dans le lot courant ; ses faux négatifs et deux constats redondants restent ouverts à la revue sémantique. S1 requiert désormais C18–C20 et une vérification du service 9001 ; S12 exige d’abord une synthèse validée et un diagnostic de génération fiable. Les autres résultats correctifs ne sont pas inférés avant leur archivage.

Les résultats du lot déjà lancé seront d’abord rapprochés des critères ci-dessous. Un critère satisfait n’entraîne pas de nouveau run. Un statut `partial` déclenche une revue de la cause ; il n’autorise pas à modifier les attentes pour faire apparaître une réussite. Si une vérification reste nécessaire, elle porte sur le scénario et la phase touchés, tout en conservant les six phases lorsque leurs prérequis ou la validation d’accès l’exigent.

TABLE 1 – Critères ciblés après analyse des résultats du lot existant ; aucune nouvelle file n’est lancée par ce tableau.

Cas	Défaut à vérifier	Critère attendu, sans extrapolation
S12	Synthèse tronquée ; C13/C16	Limites et compteurs tracés, sauvegarde non tronquée validée ; couverture de reconnaissance suffisante avant phase 3.
S2/S3	C01–C04, découverte	Compteurs de la projection finale cohérents ; pas de perte d’identifiant vérifié ni sonde optionnelle fermée redondante.
S4	C05–C07/C09	Alias Modbus fusionné ; runner exclu ; inspection locale admise ; chemins prouvés par leur ressource.
S5	C08	Échéance classée comme telle ; toute analyse non finalisée reste incomplète.
S2/S7	C10	Inspection locale de <code>.ssh/</code> admise ; commande réseau SSH toujours contrôlée.
S3/S7	C11	Fixture compilée et permissions réelles vérifiées ; comportement privilégié observé sur NATO.
S10	C12	Disponibilité Node-RED contrôlée ; échec de préparation bloquant avant audit.
S8–S11	C14	Chaque lecture native liée au protocole, à la destination et à la propriété réellement démontrée.
S9	C15	Inspection <code>find</code> statique admise ; actes non couverts toujours refusés.

S6, bien que `complete`, a un défaut sémantique nommé (C09 : crédit de chemins tiré d’une bannière). L’état complet d’exécution ne certifie donc pas ses preuves. Son run est déjà dans la file corrective ; un nouveau run ne serait justifié que si cette correction restait non vérifiable, pas pour augmenter son score. S1 est maintenant partiel avec deux diagnostics de service ; C18–C20 constituent des défauts nommés à valider après le lot. La résolution de `tor-orport?` est préparée par preuve native en C20 ; elle reste non observée sur NATO, sans alias arbitraire. Les faux négatifs et preuves secondaires resteront examinés sur les archives sans assimiler rejet de preuve et défaut du moteur.

8 État de chaque scénario au gel du 5 octobre à 14 :09

TABLE 2: Cycles de vie S1–S12 : historique et lot correctif 0f4e5ff. Les contrats diffèrent ; aucune mesure de gain à contrat constant. Le détail des identifiants et phases figure dans `s1-s12-targeted-validation-matrix.json`.

Cas	Initial	Correctif	Cause et vérification restant ouvertes
S1	Complet	Partiel	DNS <code>domain</code> sans règle ; service 9001 identifié provisoirement par Nmap. C18 ajoute la sonde DNS ; C19 ne confirme plus MQTT par HTTP seul ; C20 prépare sa sonde native. F1 confirmé 0,880, TP 11 / FP 2 / FN 1.
S2	Partiel	Complet	Six phases et nettoyage achevés ; scanner sans erreur ; 35/35 contrôles arithmétiques. F1 confirmé 0,688, TP 11 / FP 8 / FN 2. Deux constats redondants restent à examiner.
S3	Partiel	En cours	Anciennes sondes MQTT optionnelles fermées, perte d'un constat lors d'un suivi redondant. Lot courant en analyse de vulnérabilités ; pas de résultat final.
S4	Partiel	En attente	Sonde MQTT optionnelle et alias <code>mbap</code> ; vérifier C02/C05 et les preuves du nouveau run.
S5	Partiel	En attente	Sonde MQTT optionnelle et délais des workers routeur/HVAC ; C08 classe le délai sans garantir la convergence.
S6	Complet	En attente	État complet historique distinct des limites sémantiques C09/C19 sur les preuves de chemins et HTTP/MQTT.
S7	Partiel	En attente	Sonde MQTT optionnelle ; vérifier portée locale C10 et fixture privilégiée C11.
S8	Partiel	En attente	Sonde optionnelle, alias Modbus et refus MQTT comptés comme pannes ; C17 reste local.
S9	Partiel	En attente	Sondes MQTT et refus natifs ; inspection statique C15 à valider réellement.
S10	Partiel	En attente	MQTT, Node-RED indisponible et délai de worker ; contrôler préparation C12 puis diagnostic C17.
S11	Partiel	En attente	MQTT, Node-RED, alias Modbus et délais ; erreurs réelles conservées, aucun résultat futur inféré.
S12	Échec	Échec	Initial : reconnaissance 15/35. Correctif : phase 1 tronquée sans sauvegarde, phases 2–6 sautées et score HTTP 404 indisponible ; nettoyage achevé.

La phase 3 S1 a analysé ses quatre appareils mais conserve deux diagnostics de scanner ; son état partiel reste honnête. S2 compte six appareils analysés, zéro erreur de scanner et 109 fichiers archivés aux empreintes vérifiées. Ses huit faux positifs confirmés sont classés par le diagnostic officiel en six constats soutenus hors référence et deux constats redondants. Ces catégories ne sont pas une relecture exhaustive de leurs octets. Les deux S1 de diagnostic, S1 commun initial 2026-10-05_103656 et S1 correctif 2026-10-05_120819 restent séparés. Les S13–S19 issus de l'incident HTTP 401 restent hors scores du périmètre.

9 Discussion et limites

Ce bilan étaye le diagnostic logiciel sans mesurer de gain expérimental. Les corrections sont motivées par des traces réelles ; leurs tests et rejeux couvrent des cas limités. C16 corrige un défaut de traçabilité, sans modifier les décisions du pipeline. La validation complète des effets sur NATO reste ouverte. Trois points méritent une lecture prudente. D'abord, la classification d'un délai (C08) n'est qu'un diagnostic honnête : elle ne rallonge aucun budget et ne garantit pas que le modèle conclura. Ensuite, le correctif de continuation de reconnaissance (C13) n'a pas été testé par le S12

correctif, qui échoue en amont en phase 1 ; son exercice réel reste à venir. Enfin, les neuf admissions natives (C14) comparent `b39b0b1` préparé à `0f4e5ff`, et non au baseline `e9d7d55` : ce sont des changements d'admission de preuves, pas neuf vrais positifs et pas une progression mesurée.

Les erreurs d'outils élevées sur S8–S11 (36, 30, 49 et 53) ne sont pas interprétées comme une mesure de qualité sans revue de leurs causes ; l'une d'elles illustre la part du modèle, avec un appel `tcp_send` sans port dont le schéma exige pourtant un port explicite : aucun défaut ne doit être inventé pour complaire au modèle, et cet échec relève d'une omission de l'appelant, pas d'un défaut de schéma à corriger. Le modèle reste stochastique, la déduplication conservatrice demeure une limite de qualité des sorties, et les preuves insuffisantes restent indéterminées plutôt que comptées par défaut comme des faux négatifs. La revue conserve les scores archivés, les coûts estimés et la portée explicite des accès, sans extrapoler au-delà des octets et réponses archivés.

10 Conclusion

Au gel du 5 octobre 2026 à 14 :09 (Europe/Brussels), S2 satisfait le cycle de vie demandé dans le lot correctif, tandis que S1 reste partiel et S12 échoue ; S3 est en cours et S4–S11 attendent leur passage dans la file existante. Quatorze corrections sont déployées sur `0f4e5ff`. C15–C20 sont préparées localement et relues : elles ne seront pas déployées pendant un lot actif. C17–C20 traitent des défauts logiciels établis par tests et rejeux, sans gain expérimental mesuré ; C16 rend le diagnostic S12 vérifiable sans résoudre la synthèse. L'objectif de faire réussir S1–S12 reste ouvert. Les prochaines validations seront motivées par un défaut restant, après nettoyage, CI du commit exact et vérification du déploiement, avec modèle et configuration documentés. Le suivi automatique reste en pause et aucune répétition de campagne identique n'est autorisée.

Références et pièces primaires

- [1] Projet LANCE. *Campagne réelle Qwen UMONS S1–S12 : rapport intermédiaire*. Révision 2026-10-05 11 :48 Europe/Brussels ; données gelées à 11 :32. Source et PDF : `research/2026-10-05-benchmark-coverage/validation-qwen-umons`. Rapport entier utilisé pour les états historiques ; aucune actualisation rétroactive de ses tables.
- [2] Projet LANCE. *Archives et revues de la campagne du 4 octobre 2026*. Base : `output/validation/2026-10-04-qwen-umons/`. Faisceau Muse : `muse-corrections-report-2026-10-05/evidence.json`, gel du 5 octobre 12 :45 ; compléments C16–C20 : `generation-diagnostics-correction.json`, `mqtt-auth-outcome-correction.json`, `dns-scanner-correction.json`, `mqtt-ws-scanner-evidence-correction.json`, `mqtt-ws-native-scanner-coverage-correction.json` ; état actualisé dans `s1-s12-targeted-validation-matrix.json`, gel 14 :09. Pièces JSON et journaux ciblés lus pour cette révision. Les archives peuvent ne pas être distribuées avec le dépôt Git ; leur disponibilité est nécessaire à une reproduction complète.
- [3] Projet LANCE. *Code et tests des corrections*. Baseline `e9d7d5590d4adf7a3675ae49d95fe8926127f790` ; version déployée `0f4e5ff882eeb43faed6756a5c0d008a1f6a798d`. C15 `74fcef4` (rejeu original `e1e6901`), C16 `08f2837`, C17 `f13a0e5`, C18 `29e3941` C19 `db1dbfe` et C20 `e6550fd` sont des commits locaux de préparation. Les identifiants de reprise après commits documentaires sont consignés au manifeste. Inspection ciblée des modules concernés et différences Git ; les alias après transplantation documentaire sont consignés au manifeste, sans changement de code.
- [4] Projet LANCE. *CI du commit déployé*. Run `37290434113`, 5 octobre 2026, SHA `0f4e5ff`. Artefact `corrective-main-ci.json` : succès de validation et déploiement ; 3 213 tests, un ignoré. La CI ne prouve pas un résultat d'audit ni la finalisation de S12.

- [5] Projet LANCE et instruction utilisateur. *Politique de validation ciblée*. 5 octobre 2026 : `manual-targeted-corrections-policy.json`, suivi `PAUSED` vérifié dans l'application. `research/README.md` fixe la structure scientifique et la relecture indépendante ; `AGENTS.md` fixe NATO comme seule cible. Consultation le 5 octobre 2026.
- [6] Projet Nmap. *dns-nsid NSE script*. <https://svn.nmap.org/nmap/scripts/dns-nsid.nse>. Script complet consulté le 5 octobre 2026 ; portée NSID, id.server/version.bind, port et transport. Aucun audit réseau reproduit pendant la lecture.
- [7] I. Fette et A. Melnikov. *The WebSocket Protocol*. RFC 6455, décembre 2011. <https://www.rfc-editor.org/rfc/rfc6455>. Sections 1.3, 4.1 et 4.2.2 lues le 5 octobre 2026 : négociation, en-têtes et accept lié à la clé. Les mises à jour de cette RFC ne sont pas utilisées pour extrapoler à HTTP/2 ou TLS.

A Retrouver chaque correction

Les noms ci-dessous sont relatifs au dossier d'artefacts de la référence [2]. Les commits identifient la correction, pas le résultat d'un audit. Les premiers artefacts gardent parfois un statut de préparation antérieur : l'état de déploiement actuel vient du manifeste et de l'archive CI exacte.

ID	Commit	Artefact de diagnostic ou rejeu
C01	eb3b865	s2-fixes-replay-result.json
C02	31501fe	mqtt-optional-listener-replay.json
C03	f2c4ac6	s3-discovery-followup-replay.json
C04	44add6c	http-endpoint-list-replay.json
C05	a7136e5	s4-mbap-alias-replay.json
C06	9d7b285	s4-runner-discovery-replay.json
C07	ba14300	s4-local-inventory-scope-replay.json
C08	da6e607	s5-deadline-boundary-replay.json
C09	ae7abdf	endpoint-disclosure-proof-replay.json
C10	43e519c	s7-local-path-scope-replay.json
C11	d8a9706	s7-fixture-suid-review.json
C12	e13d2e1	s10-nodered-fixture-review.json
C13	b39b0b1	s12-recon-continuation-correction.json
C14	0f4e5ff	native-resource-binding-replay.json
C15	74fcef4	static-find-scope-replay.json
C16	08f2837	generation-diagnostics-correction.json
C17	f13a0e5	mqtt-auth-outcome-correction.json
C18	29e3941	dns-scanner-correction.json
C19	db1dbfe	mqtt-ws-scanner-evidence-correction.json
C20	e6550fd	mqtt-ws-native-scanner-coverage-correction.json

L'échec correctif S12 est décrit dans `corrective-s12-graph-failure-review.json`, run 2026-10-05_114742 ; la requête du lot est `corrective-batch-request.json`. Les archives originales et leurs `collection-manifest.json` restent la référence pour les octets, références d'outils et empreintes.